

Vérifications PostgreSQL — commandes idempotentes V8

Sources officielles

1. PostgreSQL, [INSERT](#) .
2. PostgreSQL, [Transaction Isolation](#) .
3. PostgreSQL, [Constraints](#) .
4. PostgreSQL, [Explicit Locking](#) .

Constats retenus

- `INSERT ... ON CONFLICT` permet de définir une action alternative face à une contrainte d'unicité. Avec `DO UPDATE`, PostgreSQL garantit, hors erreur indépendante, un résultat atomique d'insertion ou de mise à jour, y compris sous concurrence. Cette capacité convient à la réservation d'une clé d'idempotence et aux contraintes de non-duplication. 1
- Les contraintes `UNIQUE`, clés primaires et clés étrangères sont les mécanismes adaptés pour assurer les propriétés de ligne et de relation ; une contrainte `CHECK` ne doit pas tenter d'assurer une règle qui dépend d'autres lignes ou tables. Les invariants inter-objets resteront dans la frontière métier transactionnelle, avec des contraintes ciblées lorsqu'elles sont exprimables. 3
- `READ COMMITTED` est le niveau PostgreSQL par défaut. Les mutations concurrentes peuvent voir des effets récents sur la ligne concernée, mais les cas métier complexes exigent une stratégie explicite de contrôle de version, de verrouillage ciblé ou de réessai. 2
- Les transactions `SERIALIZABLE` offrent la garantie la plus stricte mais l'application doit savoir recommencer une transaction qui échoue pour cause de sérialisation (`SQLSTATE 40001`). Elles seront réservées aux invariants réellement multi-objets qui ne peuvent pas être garantis par une contrainte ou une vérification conditionnelle plus simple. 2
- Les verrous de ligne `FOR UPDATE` empêchent les écritures/verrouillages concurrents sur la même ligne jusqu'à la fin de la transaction. Les verrous explicites peuvent introduire des interblocages ; l'application doit donc acquérir les objets dans un ordre stable, garder les transactions courtes et réessayer proprement les transactions interrompues. 4

Décisions de spécification V8

- Toute commande critique portera une clé d' idempotence stable, une empreinte de requête et un résultat rejouable.
- Les règles locales seront protégées par des contraintes PostgreSQL ; les règles métier multi-objets par une transaction courte, une vérification de contexte et un contrôle de concurrence explicite.
- Une panne avant confirmation de transaction ne doit jamais être présentée comme un succès métier.
- Une répétition du même identifiant avec un contenu différent doit être refusée comme conflit d' idempotence.

Références

[1] <https://www.postgresql.org/docs/current/sql-insert.html>

[2] <https://www.postgresql.org/docs/current/transaction-iso.html>

[3] <https://www.postgresql.org/docs/current/ddl-constraints.html>

[4] <https://www.postgresql.org/docs/current/explicit-locking.html>